

Aula 5

Construção do Analisador Léxico e entrega da Etapa 1

Disciplina: Compiladores e Interpretadores
Projeto integrador: compilador da linguagem MINIC

Material de apoio baseado no Capítulo 3 de
Compiladores: Princípios, Técnicas e Ferramentas
Aho, Lam, Sethi e Ullman

1. Objetivos de aprendizagem

Ao final desta aula, você deverá ser capaz de:

- explicar como um conjunto de padrões se transforma em regras executáveis do scanner;
- aplicar a estratégia de maior casamento (*maximal munch*) e resolver prioridades entre regras;
- distinguir o reconhecimento de um lexema da ação associada ao token;
- descrever o papel do buffer de entrada, do apontador *forward* e das sentinelas;
- organizar um analisador léxico para a linguagem MINIC;
- testar, documentar e entregar a Etapa 1 de forma independente.

2. Referência no livro

O conteúdo desta aula corresponde principalmente ao Capítulo 3, especialmente às seções 3.5 — *O gerador de analisador léxico Lex* (p. 89), 3.8 — *Projeto de um gerador de analisador léxico* (p. 106) e 3.9 — *Otimização de casadores de padrão baseado em DFA* (p. 110). A paginação indicada é a impressa na edição brasileira consultada; a numeração do PDF pode divergir.

Ideia central

Um scanner combina padrões, reconhecimento, ações semânticas e controle eficiente da entrada. Ele não apenas “lê caracteres”: transforma lexemas em tokens com atributos e posições, ignorando o que não participa da estrutura sintática.

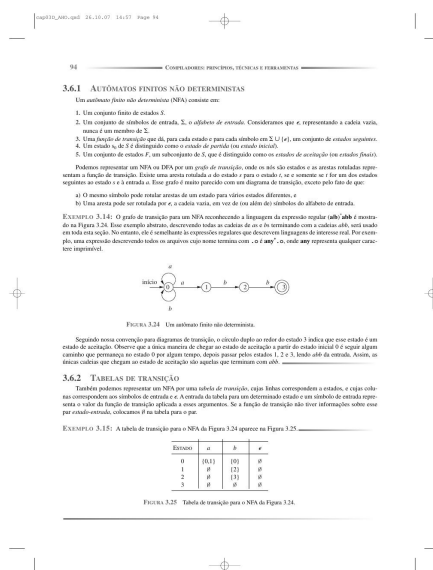


Figura de apoio do Capítulo 3 sobre o projeto de um gerador de analisador léxico — livro, p. 106.

3. Do padrão ao scanner

Na aula anterior, os tokens foram descritos por expressões regulares e autômatos. Agora esses reconhecedores precisam ser integrados em um componente operacional. O fluxo conceitual é:

fonte → buffer de entrada → reconhecedor de padrões → ação léxica → token + atributo + posição

A ação léxica pode:

- retornar um token para o parser;
- converter o lexema em um valor, como inteiro ou real;
- inserir um identificador na tabela de símbolos;
- ignorar espaços e comentários;
- registrar linha e coluna;
- emitir um diagnóstico quando nenhum padrão for aplicável.

Exemplo MINIC

Para o trecho `int total = valor + 10;`, o scanner produz uma sequência de tokens. O lexema `int` gera `INT`; `total` gera `IDENT` com atributo textual; e `10` gera `INT_LIT` com valor numérico 10.

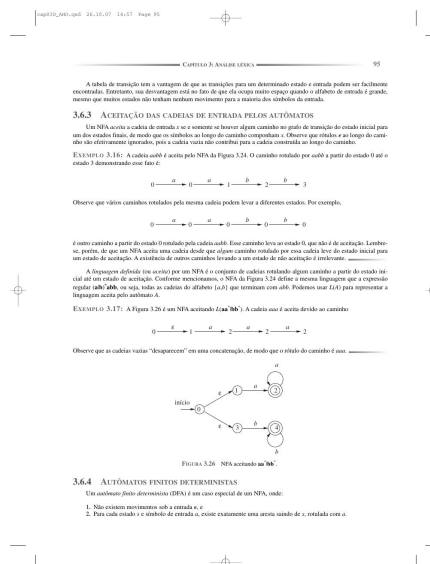


Figura de apoio do Capítulo 3 relacionada à especificação e ao reconhecimento de tokens — livro, p. 107.

4. Maior casamento e prioridade

Quando mais de uma regra pode reconhecer o início da entrada, o scanner precisa de uma política determinística. A regra usual é escolher o **maior lexema possível**. Em caso de empate, prevalece a regra que aparece primeiro.

Exemplos de conflitos

Entrada	Regras envolvidas	Decisão
==	"=" e "=="	Retornar EQ, pois o lexema de dois caracteres é o maior casamento.
<=	"<" e "<="	Retornar LE, testando o operador composto antes do simples.
if	palavra reservada e identificador	Reconhecer o identificador e consultar a tabela de palavras reservadas, ou priorizar a palavra reservada.
123abc	inteiro e identificador	Consumir 123 como inteiro e reportar o próximo lexema conforme a especificação.

Regra prática

Não aceite silenciosamente uma entrada parcial quando o restante torna o lexema inválido. O scanner deve consumir o máximo permitido pelo padrão e produzir um diagnóstico localizado quando necessário.

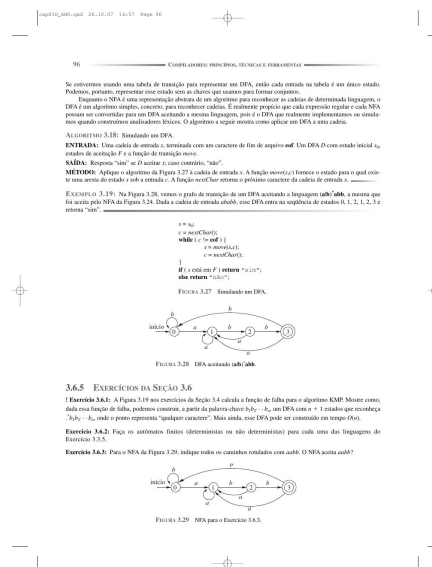


Figura de apoio do Capítulo 3 sobre reconhecimento de padrões e escolhas do analisador léxico — livro, p. 108.

5. Buffer de entrada

Ler um caractere por chamada ao sistema pode ser caro. Por isso, analisadores léxicos usam buffers para carregar blocos de entrada e apontadores para percorrê-los. O esquema de dois buffers permite que um seja processado enquanto o outro é recarregado.

Apontadores essenciais

- **lexemeBegin**: marca o início do lexema atual;
- **forward**: avança para determinar o maior casamento;
- **fim de buffer**: provoca a troca ou o recarregamento do bloco;
- **sentinela**: reduz testes repetidos para detectar o fim do buffer e da entrada.

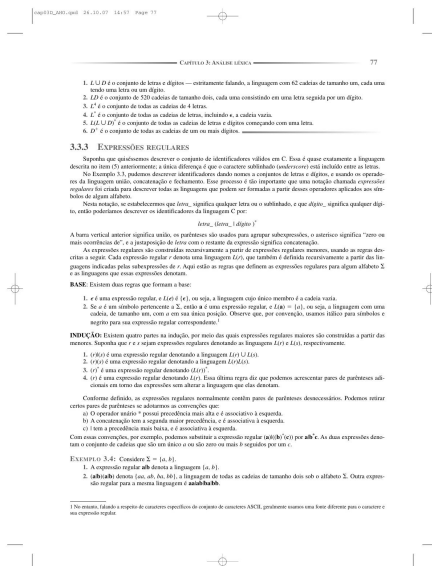


Figura de apoio do Capítulo 3 sobre buffers de entrada — livro, p. 73.

Por que isso importa no projeto?

Mesmo que a equipe utilize uma biblioteca ou gerador de scanners, compreender o buffer explica o comportamento de *lookahead*, a recuperação de lexemas e o custo do processamento. Em uma implementação manual, esse conhecimento orienta a separação entre leitura, reconhecimento e emissão de tokens.

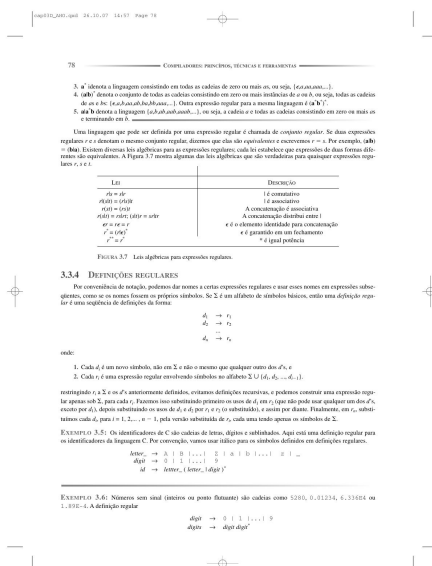


Figura de apoio do Capítulo 3 sobre sentinelas e avanço do apontador — livro, p. 74.

6. Arquitetura de um scanner para MINIC

Uma organização modular facilita os testes e permite que a Etapa 1 seja utilizada posteriormente pelo parser.

Módulo	Responsabilidade
Leitura	Fornecer caracteres, controlar buffer e manter posição.
Reconhecimento	Aplicar regras e escolher o maior casamento.
Classificação	Converter o lexema em token, palavra reservada ou identificador.
Atributos	Produzir valor numérico, texto, caractere e localização.
Erros	Detectar símbolo desconhecido, literal não terminado e lexema inválido.
Interface	Entregar ao parser token, atributo, linha e coluna.

Pseudocódigo conceitual

```

enquanto não for fim da entrada:
  ignorar espaços e comentários
  início ← posição atual
  (token, lexema) ← escolher_maior_casamento()
  se não houver casamento: emitir erro e avançar
  senão: executar ação(token, lexema, início)
  retornar EOF

```

Separação de responsabilidades

O reconhecedor decide “qual padrão casa”; a ação decide “o que fazer com o casamento”. Essa separação evita que regras de leitura, classificação e tratamento de erros fiquem misturadas.

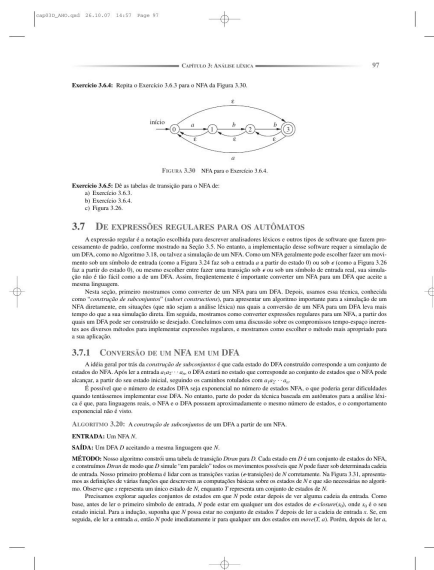


Figura de apoio do Capítulo 3 sobre a organização de um gerador léxico — livro, p. 109.

7. Especificação mínima do scanner MINIC

A tabela abaixo serve como contrato da Etapa 1. As equipes podem implementar com gerador léxico ou manualmente, mas devem manter a mesma interface e documentar extensões.

Classe	Padrão sugerido	Ação
Palavras reservadas	int float bool char void if else while for return break continue true false sizeof	Retornar token reservado.
Identificador	[A-Za-z_][A-Za-z0-9_]*	Consultar reservadas; senão retornar IDENT e atributo lex.
Inteiro	[0-9]+	Converter para valor inteiro e retornar INT_LIT.
Real	[0-9]+\.[0-9]+	Converter para valor real e retornar FLOAT_LIT.
Operadores	+ - * / % == != < > <= >= && !	Retornar token correspondente.
Delimitadores	() [] {} ; ,	Retornar token correspondente.
Espaços/comentários	espaços; //...; /*...*/	Ignorar, atualizando linhas.

Casos de fronteira

Teste especialmente: = versus ==; ! versus !=; comentários vazios; comentário de bloco sem fechamento; cadeia e caractere não terminados; número real malformato; identificador iniciado por dígito; símbolo não reconhecido.



Figura de apoio do Capítulo 3 sobre tabelas e reconhecimento de padrões — livro, p. 109.

8. Testes do analisador léxico

Uma boa suíte separa entradas válidas, inválidas e casos de fronteira. Cada teste deve informar a entrada, a saída esperada e o tipo de diagnóstico esperado.

Exemplo de saída

```
Entrada: int x = 10;

INT("int") [1:1]
IDENT("x") [1:5]
ASSIGN("=") [1:7]
INT_LIT(10) [1:9]
SEMICOLON(";") [1:11]
```

Matriz de testes

Categoria	Exemplo	Resultado esperado
Válido	int soma = a + 10;	Tokens corretos e atributos preservados.
Comentários	// comentário /* bloco */	Comentários ignorados; linha atualizada.
Operadores compostos	x <= 10 && x != 0	LE, AND, NE.
Erro léxico	x @ 2	Erro na posição do símbolo @.
Literal incompleto	char c = 'a;	Erro de literal não terminado.
Comentário incompleto	/* sem fechamento	Erro ao chegar ao fim da entrada.

Validação cruzada

Compare a saída do scanner com uma especificação manual de tokens. Se o parser ainda não estiver pronto, use um arquivo de tokens esperado como *fixture*. A Etapa 1 deve ser testável sem depender das etapas posteriores.

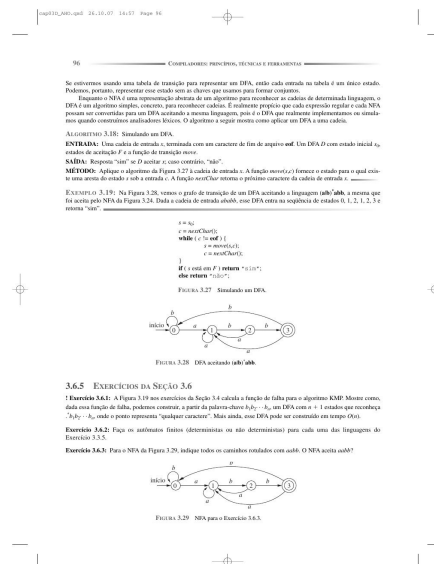


Figura de apoio do Capítulo 3 sobre implementação e testes de geradores léxicos — livro, p. 108.

9. Entrega da Etapa 1 — checklist

A entrega deve ser independente, executável e documentada.

Item	Critério de conclusão
Código-fonte	Scanner implementado e organizado, com instruções de compilação/execução.
Especificação	Tabela de tokens, padrões, prioridades e ações.
Interface	Formato documentado para token, atributo, linha e coluna.
Erros	Mensagens para símbolos inválidos e literais/comentários não terminados.
Testes	Casos positivos, negativos e de fronteira, preferencialmente automatizados.
Relatório	Decisões de projeto, limitações, exemplos de entrada e saída.
Independência	Execução possível sem parser, sem análise semântica e sem gerador de código.

Roteiro recomendado do relatório

1. Objetivo e escopo; 2. tokens e expressões regulares; 3. arquitetura do scanner; 4. estratégia de maior casamento; 5. tabela de símbolos; 6. tratamento de erros; 7. testes; 8. limitações e extensões; 9. instruções de execução.

Integração futura

Na Etapa 2, o parser consumirá a mesma sequência de tokens. Por isso, não altere nomes ou atributos sem registrar a decisão. O scanner deve ser uma unidade estável, com contrato claro e testes que possam ser reutilizados durante a integração.

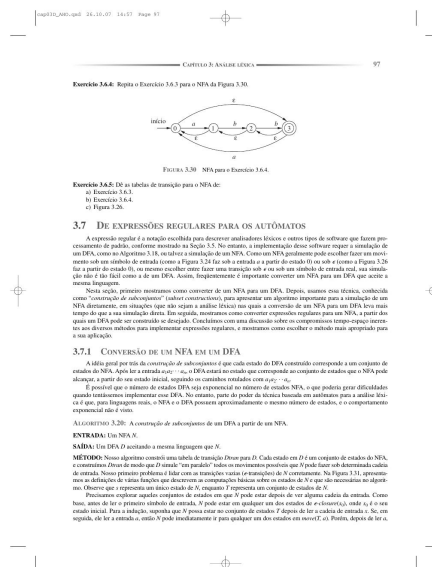


Figura de apoio do Capítulo 3 sobre a composição de reconhecedores — livro, p. 109.

10. Exercícios e síntese

Exercícios de fixação

1. Explique por que `==` deve ser testado antes de `=`. 2. Descreva a política de desempate entre regras de mesmo tamanho. 3. Proponha uma estrutura para armazenar linha, coluna, lexema e valor. 4. Dê três casos em que o scanner deve emitir erro. 5. Explique como dois buffers e sentinelas reduzem o custo de leitura. 6. Escreva um teste para diferenciar palavra reservada e identificador.

Desafio de implementação

Implemente um scanner mínimo para as expressões e comandos da MINIC. Em seguida, acrescente: (a) comentários; (b) operadores compostos; (c) linha e coluna; (d) recuperação após erro. Registre cada extensão no relatório.

Resumo

- o scanner é a ponte entre caracteres e tokens;
- o maior casamento resolve lexemas com prefixos comuns;
- prioridade e consulta a palavras reservadas resolvem empates;
- buffers, *lookahead* e sentinelas contribuem para eficiência;
- a Etapa 1 deve funcionar e ser avaliada de modo independente.

Referência principal

AHO, Alfred V.; LAM, Monica S.; SETHI, Ravi; ULLMAN, Jeffrey D. *Compiladores: Princípios, Técnicas e Ferramentas*. 2. ed. São Paulo: Pearson, 2008. Capítulo 3 — Análise léxica: seções 3.1–3.9, especialmente pp. 70–119. As imagens reproduzidas são excertos de apoio didático do exemplar fornecido pelo usuário; consulte o livro para o contexto completo, definições formais e exercícios originais.